thus instructs the node to propagate to its new position. If the node reaches its destination, it will pause there; or it may also happen that the simulation ends before. In that case, the node will not be able to reach the destination. Also, sometimes a new destination is set during a course to a location; in that scenario, the node will change course to the new destination in between.

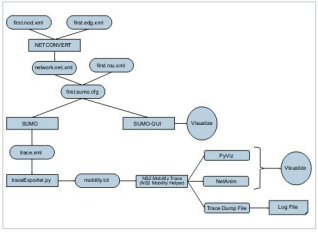

Figure 1: Hierarchy of the integration of SUMO files with ns-3

SUMO

Simulation of Urban Mobility (SUMO) is an open source traffic simulation package that includes net-import and demand modelling components. Many research topics, like the route choice and the simulation of vehicular communication or traffic light algorithms, are studied with the help of SUMO. The framework of SUMO serves in various projects like traffic management strategies or to simulate automatic driving. Simulations in SUMO comprise time-discrete and space-continuous vehicle movements, multi-lane highways and streets, traffic lights, Open GL for the graphical interface and fast execution speed. SUMO is edge-based, can operate with other applications at run time, has portability and detector-based outputs.

The flow of the integration of SUMO traces with ns-3 for mobility provisioning is shown in Figure 1.

Constructing a scenario in SUMO

In order to generate traffic with the help of the SUMO simulator, a scenario or network needs to be created. SUMO Street Network consists of nodes (junctions) and edges (streets connecting the junctions). SUMO Simulator requires *.nod.xml* and *.edg.xml* to define the junctions and streets joining them.

The *nod.xml* file contains the location (*x* and *y* coordinates) of junctions. An example is given below:

```
first.nod.xml

<nodes>
    <node id="1" x="-300.0" y="5.0" />
    <node id="2" x="+300.0" y="5.0" />
    <node id="3" x="5.0" y="-300.0" />
    <node id="4" x="5.0" y="+300.0" />
    <node id="5" x="5.0" y="5.0" />
</nodes>
```

To join the above nodes, edges are defined in the *edg.xml* file using the target node ID and source node ID. An example is given below:

```
first.edg.xml

<edges>
    <edge from="1" id="A" to="4" />
    <edge from="4" id="B" to="2" />
    <edge from="2" id="C" to="5" />
    <edge from="5" id="D" to="3" />
    <edge from="2" id="E" to="3" />
</edges>
```

To build a network, the above defined node file and edge file are required. And using the *netconvert* utility, a *net.xml* file is generated as follows:

```
$netconvert -n first.nod.xml -e first.edge.xml -o network.net.xml
```

Vehicles in the traffic are defined with route data in the *rou.xml* file. For example:

```
first.rou.xml

<routes>
<vType id="BoogyA" length="5.75" maxSpeed="90.0" sigma="0.4" />
<vType id="BoogyB" length="7.5" maxSpeed="60.0" sigma="0.7" />
  <route id="rou01" edges="A B C D"/>
  <route id="rou02" edges="A B E"/>
  <vehicle depart="0" id="v0" route="rou01" type="BoogyA"
color="1,0,0" />
  <vehicle depart="1" id="v1" route="rou02" type="BoogyA" />
  <vehicle depart="2" id="v2" route="rou01" type="BoogyA" />
  <vehicle depart="3" id="v3" route="rou02" type="BoogyA" />
  <vehicle depart="5" id="v4" route="rou01" type="BoogyA" />
  <vehicle depart="6" id="v5" route="rou02" type="BoogyA" />
  <vehicle depart="7" id="v6" route="rou01" type="BoogyA" />
  <vehicle depart="8" id="v7" route="rou02" type="BoogyA" />
  <vehicle depart="9" id="v8" route="rou01" type="BoogyA" />
  <vehicle depart="11" id="v9" route="rou02" type="BoogyA" />
  <vehicle depart="14" id="v10" route="rou01" type="BoogyA"/>
  <vehicle depart="16" id="v11" route="rou01" type="BoogyA"/>
  <vehicle depart="17" id="v12" route="rou01" type="BoogyB"
color="1,0,0" />
  <vehicle depart="18" id="v13" route="rou02" type="BoogyB" />
  <vehicle depart="19" id="v14" route="rou02" type="BoogyB" />
```